DIGITAL ELECTRONICS

* Number System:-

— Base and their group of Number System:-

N.S.	Base	Base set
Binary	2	0,1
octal	8	0,1,2,3,4,5,6,7
Decimal	10	0,1,2,3,4,5,6,7,8,9
Hexa-Decimal	16	0,1,2,3,4,5,6,7,8,9, 10-A, 11-B, 12-C, 13-D, 14-E, 15-f

N.S.	min/max no. of 4 Digit
Binary	min = 0000 (common) max = 1111
octal	max = 7777
Decimal	max = 9999
Hexa-Decimal	max = ffff.

(1) Decimal → Binary:-

Eg. 1. $(25)_{10} = (?)_2$

2	25	1
2	12	0
2	6	0
2	3	1
	1	

$= (11001)_2$.

(2) Decimal Fraction to Binary:-

Eg: $(0.65625)_{10} = (?)_2$

→ $0.65625 \times 2 = 1.31250 = 1$

→ $0.31250 \times 2 = 0.62500 = 0$

→ $0.62500 \times 2 = 1.2500 = 1$

→ $0.2500 \times 2 = 0.500 = 0$

→ $0.500 \times 2 = 1.000 = 1$

$= (0.10101)_2$

(3) Binary to Decimal:-

Eg: $(11001)_2 = (?)_{10}$

$1\ 1\ 0\ 0\ 1$

$2^4\ 2^3\ 2^2\ 2^1\ 2^0 = 16+8+1 = (25)_{10}$

(× × under 2^2 and 2^1)

(4) Binary Fraction to Decimal:-

Eg: $(0.10101) = (?)_{10}$

→

$1\ 0\ 1\ 0\ 1$

$0.50 + 0.25 + 0.125 + 0.0625 + 0.3125$

$= (0.65625)_{10}$

2^{-1}	$= 0.50$
2^{-2}	$= 0.25$
2^{-3}	$= 0.125$
2^{-4}	$= 0.0625$
2^{-5}	$= 0.03125$

(5) Octal to Binary:-

octal	Binary.
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Eg: $(642)_8 = (?)_2$.

6 4 2

110 100 010

$= (110100010)_2$.

→ Each digit with 3 bit.

(6) Binary → octal:-

→ Pair of ~~three~~ 3-3-digit from right to left.

Eg: $(1011001)_2 = (?)_8$.

←

001 011 001

$= (131)_8$.

(7) Octal fraction to Binary:-

→ Same method as octal to Binary.

Eg: $(267.15)_8 = (?)_2$.

2 6 7 . 1 5

010 110 111 001 101

$= (010110111.001101)_2$.

(8) Binary fraction → octal :-

→ Same method as Binary → to octal.
→ Normal digit right to left
→ After (.) left to right.

Eg: $(1001101.1001)_2 = (?)_8$.

001 001 101 . 100 100 $= (115.44)_8$.
1 1 5 . 4 4

(9) octal → to Decimal :-

Eg: $(7423)_8 = (?)_{10}$

$7 + 4 + 2 + 3$
$8^3 \; 8^2 \; 8^1 \; 8^0$ $= 3584 + 256 + 16 + 3 = (3859)_{10}$

(10) Decimal → to octal :-

Eg: $(3859)_{10} = (?)_8$

8	3859	3
8	482	2
8	60	4
	7	

$= (7423)_8$.

(11) Octal → Decimal ..

Eg: $(247.54)_8 = (?)_{10}$

$2 + 4 + 7 \;.\; 5 + 4$

$8^2 \; 8^1 \; 8^0 \;.\; 8^{-1} \; 8^{-2}$

$64 \; 8 \; 1 \;.\; 0.125 \; 0.015625$

$= 128 + 32 + 7 \;.\; 0.625 + 0.0625$

$\Rightarrow (167.6875)_{10}$.

$8^{-1} = 0.125$

$8^{-2} = 0.015625$

$8^{-3} = 0.001953125$

(12) Decimal fraction → Octal ..

Eg $(167.6875)_{10} = (?)_8$.

8	167	7
8	20	4
	2	2

$\Rightarrow (247)_8$

$0.6875 \times 8 = 5.5000$

$0.5000 \times 8 = 4.0000$

$\Rightarrow (167.6875)_{10} = (247.54)_8$

(13) Hexadecimal → Binary :-

→ Each digit → 4 binary bit.

Hexa	Binary
0	0000
1	0001
2	0010
3	0100
4	0101
5	0110
6	0111
7	1000
8	1001
9	1010
10 A	1011
11 B	1100
12 C	1101
13 D	1110
14 E	1111
15 F	

eg:- $(7A2)_{16} = (?)_2$.

7 A 2

0111 1010 0010 $= (011110100010)_2$.

(14) Binary → to Hexadecimal :-

q: $(11010\,1111)_2 = (?)_{16}$

0001 1010 1111

1 10 15 $= (1AF)_{16}$.

(15) Hexadecimal fraction → to Binary :-

→ $(175.5)_{16} = (?)_2$.

= 1 7 2 . 5

0001 0111 0010 . 0101 $= (101110010.0101)_2$.

(16) Binary fraction → to Hexadecimal :-

q: $(101110010.11)_2 = (?)_{16}$.

0001

1 7 2 . 1100

12

C $= (172.C)_{16}$.

(17) Hexadecimal → to Decimal :-

eg: $(549)_{16} = (?)_{10}$

$\Rightarrow 16^2\ 16^1\ 16^0 = 256 + 16 + 1$

$= 1280 + 160 + 9 = (1449)_{10}$.

(18) Decimal → Hexadecimal :-

eg: $(2046)_{10} = (?)_{16}$.

16	2046	14
16	127	15
	7	7

$\Rightarrow (7FE)_{16}$.

(19) Hexadecimal fraction → Decimal :-

eg: $(7EA.A)_{16} = (?)_{10}$

$\Rightarrow 256 \cdot 16\ 1 \cdot 0.0625 \Rightarrow (2026.625)_{10}$.

(20) Decimal fraction → Hexadecimal :-

eg: $(2026.625) = (?)_{16}$

$.625 \times 16 = 10.00$

16	2026	10
16	126	14
	7	

$\Rightarrow (7EA)_{16} \Rightarrow (7EA.A)_{16}$.

(21) Hexa → octal :- 8DA. ←

1000 1101 1010

4 3 3 2 $\Rightarrow (4332)_8$.

(22) octal → Hexa :- $(377)_8$

011 111 111

0 15 15 $= (FF)_{16}$.

(23) octa fraction → Hexadecimal

$(1257.5)_8 = (?)_{16}$

$\Rightarrow$ 001 010 101 111 . 101

$\Rightarrow$ 2 10 15 10 $\Rightarrow (2AF.A)_{16}$.

* Binary operation:-

1) Add:
$0 + 0 = 0$
$0 + 1 = 1$
$1 + 0 = 1$
$1 + 1 = 0$ (carry 1)

eg: $1011.01 + 1100.11$
$\rightarrow (11000.00)_2$

2) Sub:
$0 - 0 = 0$
$1 - 1 = 0$
$1 - 0 = 1$
$0 - 1 = 1$
Next digit borrow.

3) Sign magnitude representation:-

B_4 B_4 B_3 B_2 B_1

Sign

0 (+ve) 1 (-ve)

4) 1's Compliment:-

→ Inverse of number.
→ Or Subtract 1.

Q. 1001101
(0110010) ← 1's Complement

(5) 2's Complement :-

→ Add 1 in 1st complement of Binary digit.

Eg. (1001101)

⇒ 0110010
+1
(0110011) ← 2nd Complement.

(6) 8th Complement :-

→ only Octal.

→ minus 7 and then 1 Add.

2107
+1
2110

Eg. 2107

⇒ 7777
2107
5670
+1
5671 ← 8th Complement



* Binary Codes :-

(1) Binary → BCD, Excess-3, Gray code.

(2) Non-Binary → Morse code.

(3) Alphanumeric → ASCII, EBDIC, Antenna code (8 Bit)
(7 Bit + 1 Parity) 8 Bit

(1) BCD code :-

→ Binary Coded Decimal.
→ Also called 8421 code
→ 4-bit binary code.
→ Decimal digit 0-9 are represented in binary.
→ Weighted Code (weight 8, 4, 2, 1).
→ Not a Self-Complementary code.
→ For BCD addition :-
- If the sum of two digit in BCD is ≤ 9, the result is already correct.
- If greater than 9, add 6 (0110) to get correct BCD result.

* Decimal to Binary Conversion :-

Decimal	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Eg: $(943)_{10}$ = BCD.

=> 100101000011.

Weighted code :-

→ Each digit has fixed weight

→ Eg. BCD, 2421 code.

Non-Weighted code :-

→ Digit dont have any fixed weight.

→ Ex. Excess-3, Gray code.

(2) Excess-3 Code :-

→ 4 Bit code

→ Non Weighted.

→ Add +3 (0011) to get

→ Self Complimentary code.

Binary	Excess-3
0 — 0000	0011
1 — 0001	0100
2 — 0010	0101
3 — 0011	0110
4 — 0100	+ 0011 0111
5 — 0101	1000
6 — 0110	1001
7 — 0111	1010
8 — 1000	1011
9 — 1001	1100

Self complement.

Ex. $(732)_{10}$:-

~~Sol~~ : 7 3 2

0111 0011 0010

BCD = 011100110010

Excess-3 : 0111 0011 0010

+ 0011 0011 0011

1010 0110 0101

(3) Gray code

→ Non-weighted code.
→ only 1 Bit change. i.e minimum change code.
→ Reflected code
→ Unit Distance code.
→ minimum error code
→ EX-OR use.

Ex. 1:- $(11010)_2$ = Gray code

11010 = 10111.

↓

10111

Ex. Gray to Binary :- $(11011)_{Gray}$

1 1 0 1 1 = $(10010)_2$.

↓ ↓ ↓ ↓ ↓

1 0 0 1 0

(4) Alphanumerical Codes :-

→ Codes that represent alphabets, numbers & Punctuation marks.
→ Eg. ASCII & EBCDIC
→ Normal ASCII is 7-bit, Extended is an 8-bit code.
→ American Standard code for Information Interchange
→ Extended Binary Coded Decimal Interchange code
→ EBCDIC – 8-bit code.

* Important theorem :-

(1) Duality :- AND ⟷ OR
0 ⟷ 1
1 ⟷ 0

(2) D-morgan's

(3) Boolean Algebric theorem :-

Name	AND form	OR form.
Identity	$1 \cdot A = A$	$0+A = A$
Null	$0 \cdot A = 0$	$1+A = 1$
Idenbotent	$A \cdot A = A$	$A+A = A$
Inverse	$A \cdot \bar{A} = 0$	$A+\bar{A} = 1$
Commutative	$A \cdot B = B \cdot A$	$A+B = B+A$
Associative	$(AB)C = A(BC)$	$(A+B)+C = A+(B+C)$
Distributive	$A+BC = (A+B)(A+C)$	$A(B+C) = AB+AC$
Absorbtion	$A(A+B) = A$	$A+AB = A$

* Logic gates:-

→ Base of digital electronics.

→ 3 types

(i) Basic gates → AND, OR, NOT.

(ii) Universal gates → NAND, NOR

(iii) Special purpose gate → EX-OR, EX-NOR.

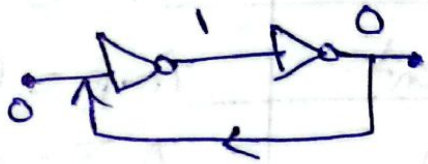
(1) Basic gates:-

(A) NOT Gate:-

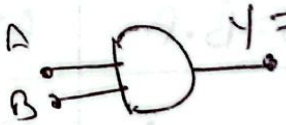


A	$\overline{A}$
0	1
1	0

→ Bistable multivibrator :-



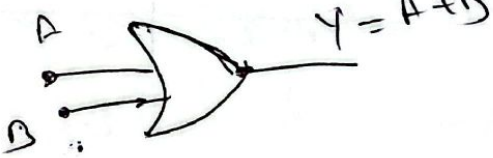
(B) AND Gate :-



A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

AND = Series
OR = Parallel

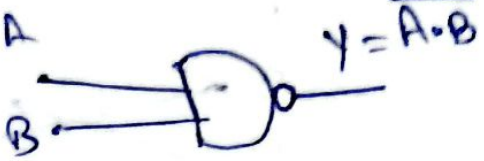
(C) OR Gate :-



A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

(2) Universal Gate :-

(A) NAND Gate :-



→ AND + Not.

A	B	$Y = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

→ Bubble OR Gate = NAND Gate.

(B) NOR Gate :-

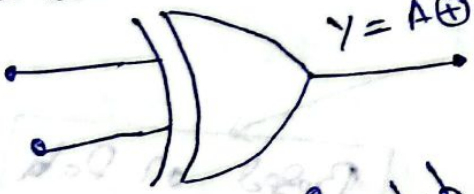
→ OR + Not.

→ Bubble AND Gate = NOR Gate.

A	B	$Y = \overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

(3) Special purpose Gate :-

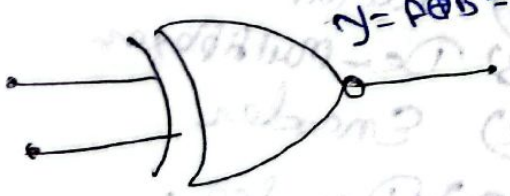
(A) EX-OR Gate :-

→ 

→ Staiy-case Switch.

A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

(B) EX-NOR Gate :-



A	B	$Y = \overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1

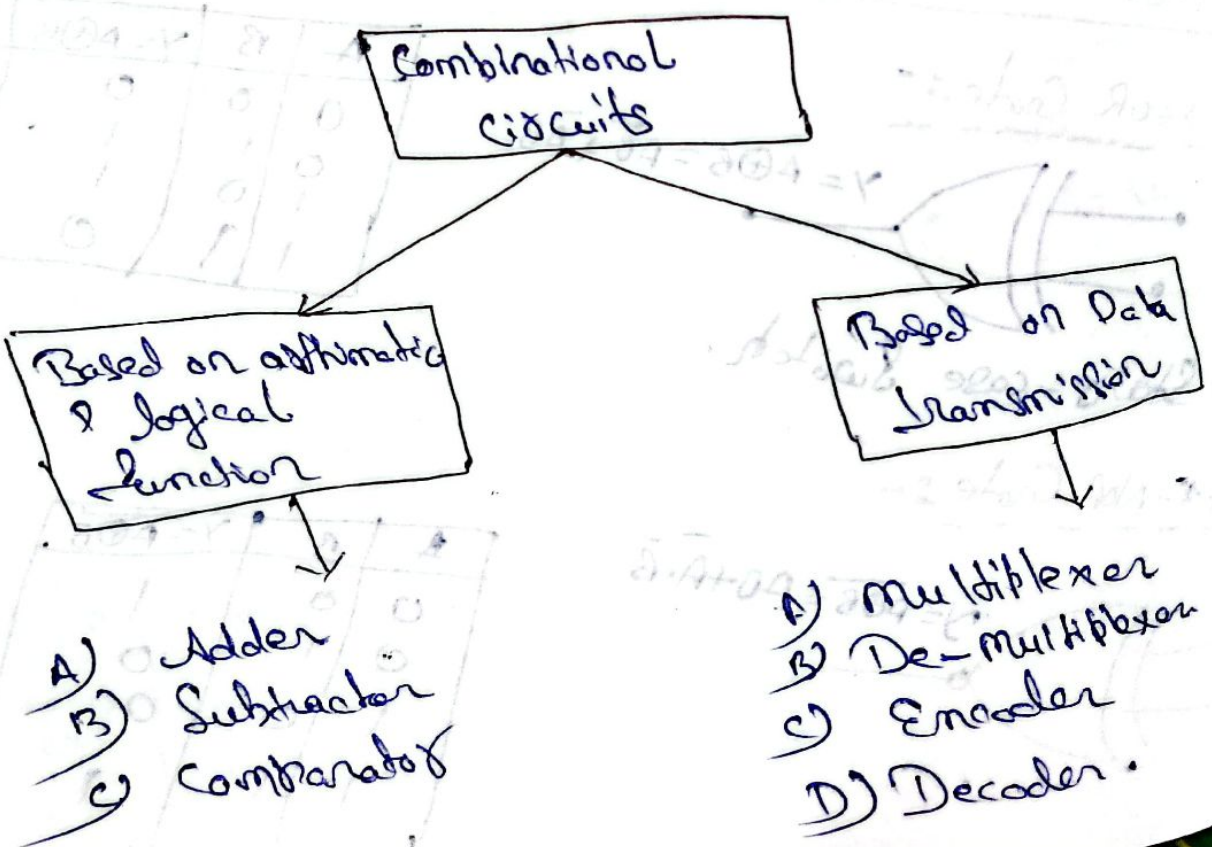
**

Logic Gate	No. of NAND	No. of NOR
NOT	1	1
AND	2	3
OR	3	2
Ex-OR	4	5
Ex-NOR	5	4

* Combinational Circuits :-

→ output depends on present input.
→ No dependency on feedback, memory, or timing.
→ Built using logic gates.
→ Does not contain any memory elements.

* Classification of Combinational Circuits :-



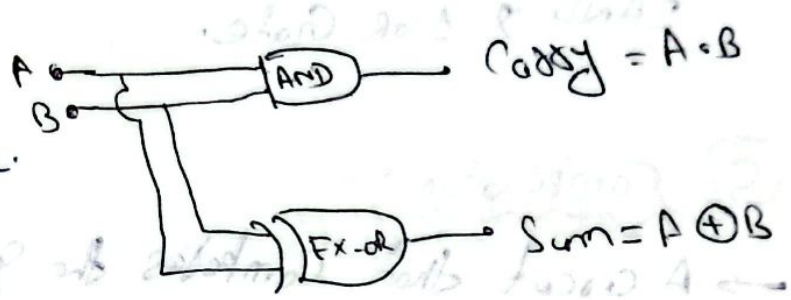
* Based on Arithmatic functions:

(A) Adder:-

→ use for binary no. addition.

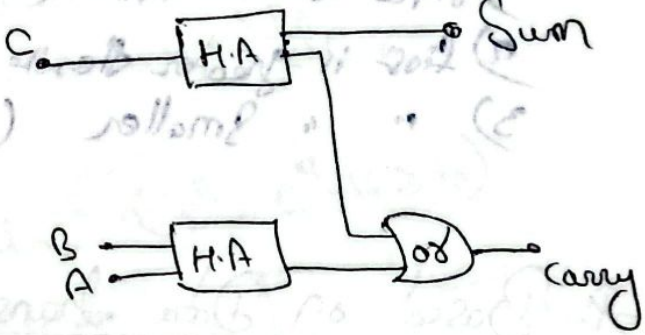
(1) Half adder:-

→ 2 input & 2 output.
→ 1 And & 1 Ex-OR Gate.
→ output of AND = Carry.
→ output of Ex-OR = Sum.
→ 5 NAND Gate or 5 NOR Gates.



(2) Full adder:-

→ 3 input & 2 output.
→ (3 input = 2 inp & 1 carry)
→ 2 half adder & 1 OR Gate.
→ output = Sum & Carry.
→ 9 Nand Gates or 9 NOR Gates.

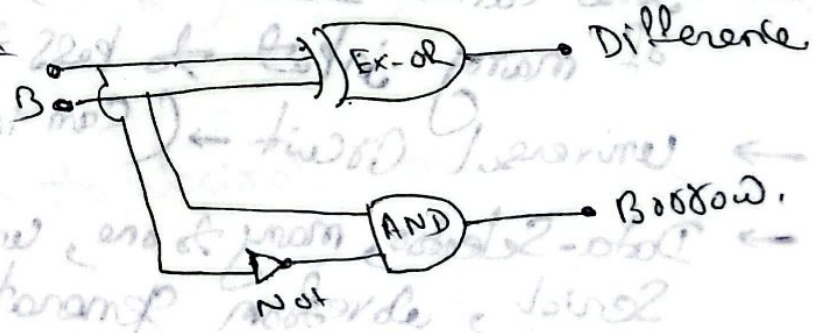


(B) Subtractor:-

→ use for Substraction.
→ 2 output = Borrow & Difference.
→ Half Sub & full subtractor.

(1) Half Subtractor:-

→ 2 input & 2 output.
→ 1 Ex-OR, 1 AND, 1 NOT.

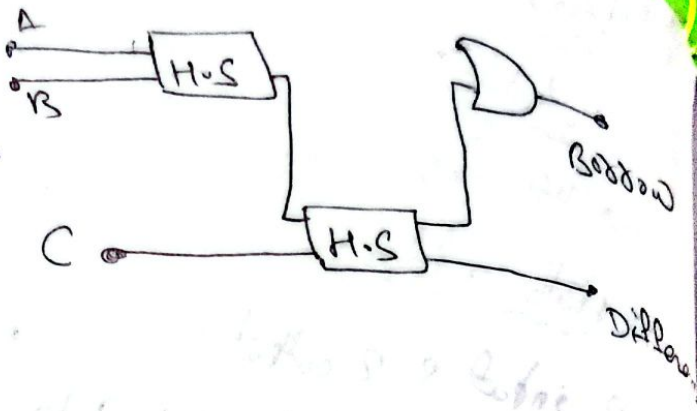


② Full Subtractor :-

→ 3 input & 2 output.

→ 2 Half Subtractor + 1 OR Gate.

→ total 7 Gates = 2 EX-OR, 2 AND & 1 OR Gate.

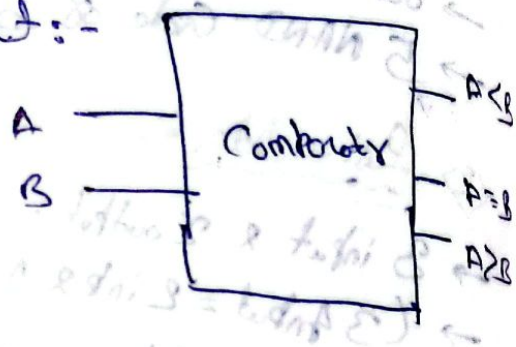


ⓒ Comparator :-

→ A circuit that compares the given inputs and produces an output based on that comparision.

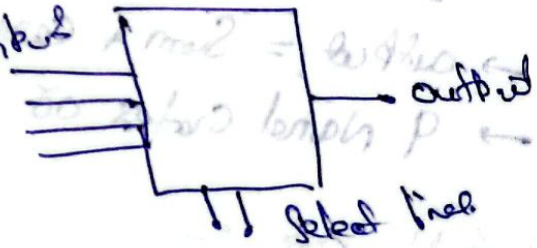
→ It compares the inputs and gives output :-

1) Input are the same (A == B)
2) first is greater than the second (A > B)
3) " " smaller (A < B)



* Based on Data Transmission :-

(A) Multiplexer :-



→ many to one principle.

→ multiple input, one output.

→ No. of input depends on the number of select lines with n select lines, there are 2^n input lines.

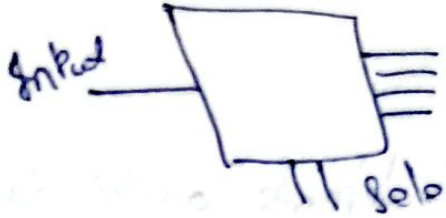
→ Also called Data-Selector, because it selects only one of many input to pass to the output.

→ Universal Circuit → (Can implement many kind of circuit)

→ Data-Selector, many to one, Universal logic converter, parallel to Serial, Waveform generator.

→ 2×1 mux, 4×1, 8×1 mux.

(b) De-multiplexer :-

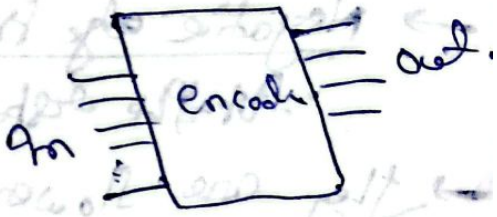


→ many to one principle.
→ Reverse operation of a mux
→ one to many, data distributor, serial & parallel. ($n = \log_2 m$)
→ Cannot be used as a universal gate.
→ 1×2 Demux, 1×4 Demux etc.

(c) Encoder :-

→ A circuit with multiple inputs and multiple outputs, but the no. of input greater than the no. of outputs.
→ Size : $2^n \times n$ (where 2^n = no. of inputs, n = no. of outputs).
→ Used to convert various code into binary form.
→ Also used as decimal to BCD encoders. etc.

or Octal to binary → 8×3 encoder.
Decimal to binary → 10×4 encoder. ($2^n = 2^4 = 16 \geq 10$)
Hexa to binary → 16×4 encoder



(d) Decoder :-

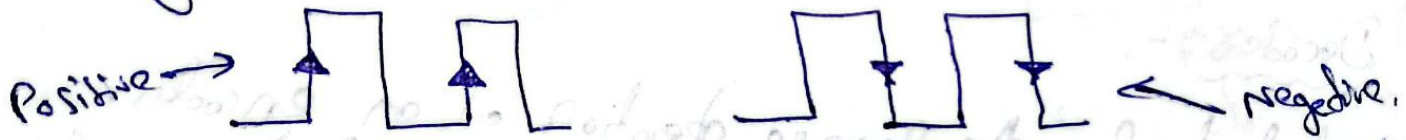
→ Used to perform the reverse operation of an encoder.
→ Has multiple input and multiple output, but the no of output greater than no. of input.
→ Size $n \times 2^n$ (n = input, 2^n = output)
→ BCD Decoder.
→ Converting binary number to decimal form.
→ 3×8 decoder, 4×16 decoder.

* Sequential logic Circuit :-

→ output depends on the current input and previous output / State.
→ needs memory → to store the previous State / information.
→ Slower than Combinational Circuit.
→ uses feedback.
→ two types.

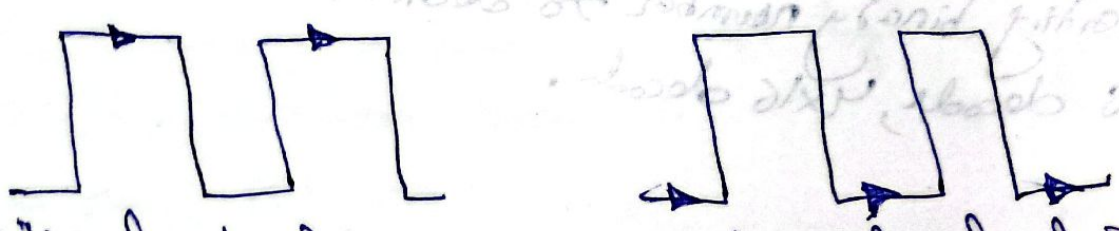
(1) Synchronous Sequential Circuit :-

→ Their memory elements are clocked flip-flops.
→ They are edge triggered. (State changes only when a clock edge occurs).
→ Positive edge trigger :-
  • flip-flop responds on the rising edge of clock (low → high)
→ Negative edge trigger :-
  • flip-flop responds on the falling edge of clock (high → low)
→ They are slower compared to async circuit.



(2) Asynchronous Sequential Circuit :-

→ Their memory element are unclocked flip-flop or time delay.
→ level triggered.
→ Because of clock pulse absence, it's faster than Sync ckt.



* Latches :-

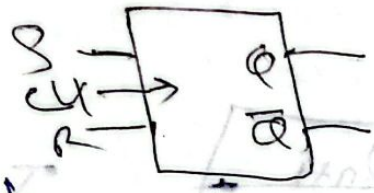
→ Asynchronous input.
→ clock pulse use ✗
→ Temporary Storage.
→ level trigger need.
→ low power consume.

* flip-flop :-

→ Synchronous input.
→ Clock pulse use.
→ Permanent Storage.
→ Edge trigger need.
→ more power consume.
→ logic gates + combinational circuit.
→ Data Transfer, data Storage & frequency divider.
→ 2 inputs & 2 output.
→ output Q & $\overline{Q}$
→ 1 bit Store.

Types :

(A) S-R flip flop :-



S	R	Q(next)	Action
0	0	Last Q	No change
0	1	0	Reset
1	0	1	Set
1	1	Invalid	Not permitted

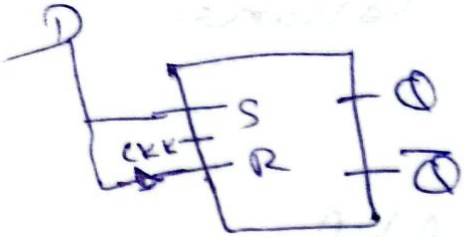
→ Called as latch flip flop.
→ can be made using NAND or NOR.
- Using NOR → active high SR-flip flop.
- Using NAND → active low SR flip flop.

→ Has two input : S(Set), R(Reset), Pulse clock (Also clocked SR ff)
→ S=R=1, the condition is not permitted/invalid (J-K ff used)
→ S=R=0, it holds previous (memory) state.

(B) D - flip flop

- D - Delay.
- → A modified form of the SR ff with only one input terminal D.
- → Mainly used for building Registers.
- → Called as Data transmission ff or transparent latch ff.

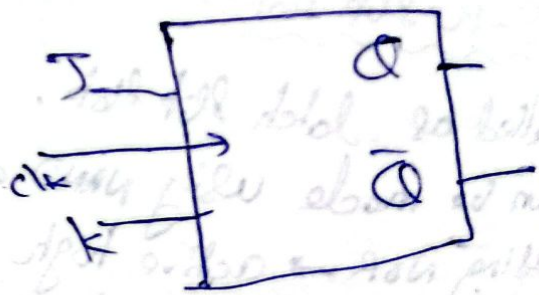
Clk	D	Next state (Q_{n+1})
0	X	last state (Q_n)
1	0	0
1	1	1



(C) J-K flip flop :-

- → Remove the invalid state of the SR flip-flop.
- → Has the same Set, Reset, No change condⁿ as S-R ff.
- → when J=K=1, called as race/race around condition because output keep toggling.
- → To eliminate race around, a master slave ff is used.
- → JK → Jack kilby or (Jump key ff).

Clk	J	K	Q_{n+1}
X	0	0	Q_n (prev state)
1	0	0	Q_n (prev state)
1	0	1	0 (Reset)
1	1	0	1 (Set)
1	1	1	$\bar{Q}_n$ (toggle)



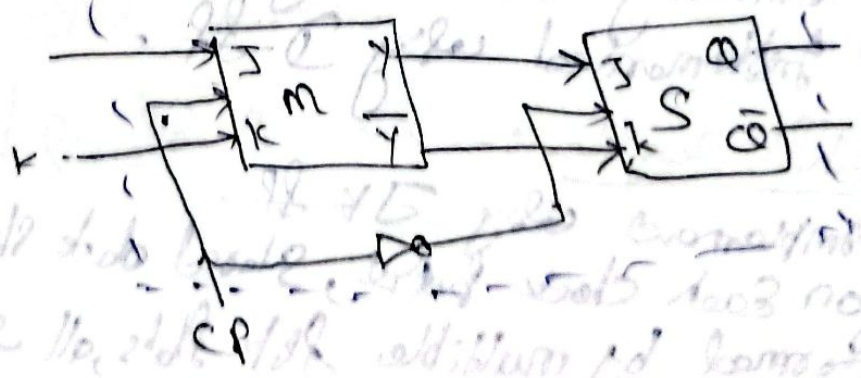
* Race around Condition :-

→ J=K=1 occurs toggle mode.
→ Happen only in level-triggered FF.
→ clock pulse width > propagation delay.
→ $T_{pw} > T_{pd}$, the output toggles many times within one clock pulse → race around condition.
→ To avoid it, make the clock pulse width less than the propagation delay ~~T_{pw}~~ ($T_{pw} < T_{pd}$) or use an edge triggered / master slave flip flop.

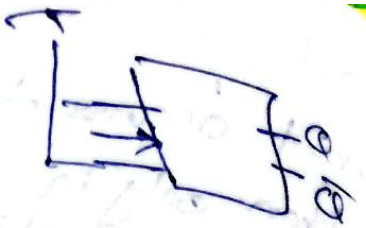
T_{pw} = clock pulse width, T_{pd} = Propagation delay.

① J-K master slave flip-flop :-

→ Used to remove the race-around Condⁿ.
→ Contain two J-K FF : master & slave.
→ The master is triggered on the positive ~~side~~ edge of the clock and slave on the negative edge.
→ Because the slave is edge-triggered and follows the master only on the next edge, multiple toggling during one clock pulse does not occur.
→ The final output Q changes only when the slave changes, so Circuit is stable.



(E) T-flip flop:



- → also called Toggle flip flop.
- → modified form of the JK ff.
- → Input J & K are tied together and replaced by a single input T.

Clk	T	Q_{n+1} (next state)
0	0	Q_n (no change)
1	0	Q_n (Hold)
↓	1	Q_n (toggle)

* Registers:-

- → A logical circuit used to store binary information and control its movement
- → Built from multiple flip-flop that are logically connected.
- → To store N bits, you need N ff.
- → Information can be stored and retrieved in serial or parallel form.
- → Serial data is temporary, while parallel data is called positional data.
- → Types:

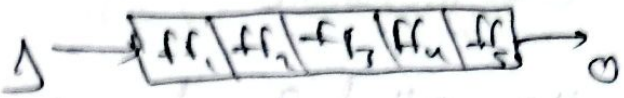
(A) Buffer Register:-

- → Temporary store binary data.
- → Implemented using D-ff.

(B) Shift Register:-

- → Implemented using JK ff.
- → On each clock pulse, stored data shift by one bit.
- → Formed by multiple flip-flops, all receiving the same clock

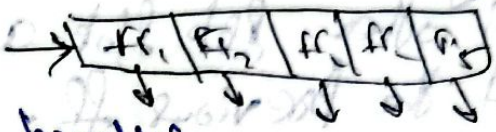
Types:



(i) SISO - Serial in Serial out :

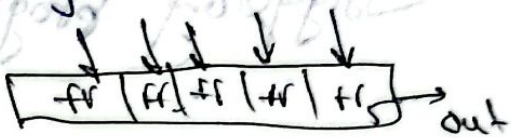
→ Data is entered into the register serially, one bit at a time,
→ Stored info is taken out serially.
→ Because bit go on and come out one by one, it is slow speed

(ii) SIPO - Serial in parallel out.

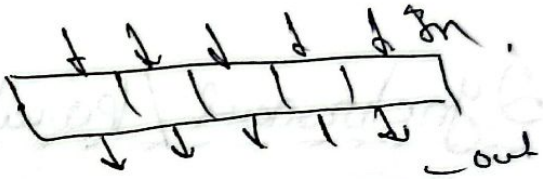


→ Entered serially, taken out parallelly.

(iii) PISO - Parallel in serial out :-



(iv) PIPO - Parallel in parallel out :-



(v) Bi-directional :-

→ Data can be shifted in both direction (left or right)
→ A control signal selects the direction.

Control = 0 → data shift right
Control = 1 → data shift left.

* Counter :-

→ A circuit used to count clock pulses.
→ two types :-

1) Asynchronous / ripple counter
2) Synchronous / Parallel counter.

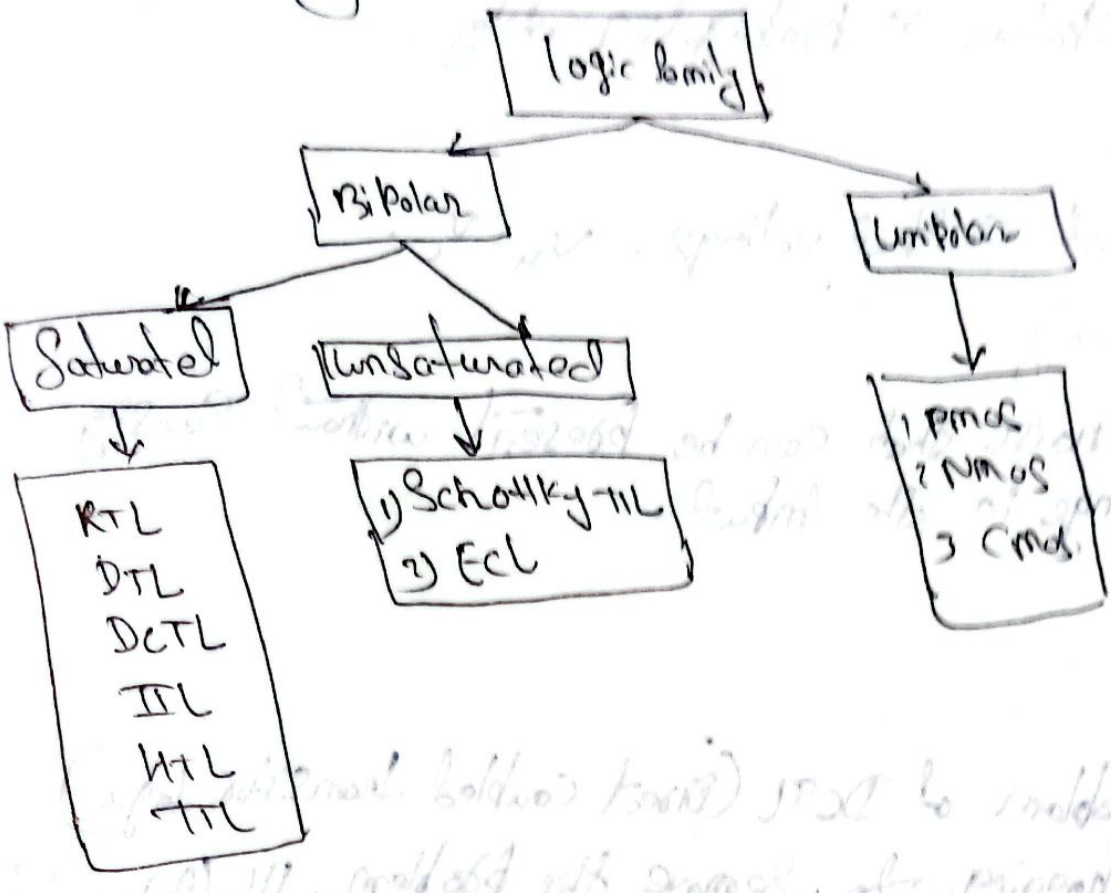
(1) Asynchronous / Ripple Counter:-

- → Also called Serial counter because the flip-flop are arranged in serial.
- → Slower speed.
- → Only the LSB flip flop receive the clock directly.
- → All other flip flop are triggered from the output of the previous ff.
- → Triggering progresses like a water ripple.

(2) Synchronous / Parallel Counter:-

- → All ff are triggered by the same clock pulse
- → Connected in parallel.
- → High speed.
- → More Complex Circuit.

* Logic family.



→ Characteristics of logic family:-

(1) Speed :- The time taken between a change in the input and the corresponding change in the output (propagation delay)

(2) Fan-out :- The number of loads (inputs of other gates) that the output of a gate can drive

(3) Fan-in :- The max number of inputs that a logic gate can have.

(4) Power dissipation :- Energy used by each gate, (mW)

(5) Propagation Delay :- (ns)
Time taken for a signal to travel from input to output.

(6) Noise margin :- The max. noise voltage that can be tolerated without causing any problem in the correct operation of the circuit.

(7) Figure of merit :-

→ Power dissipation x propagation delay.

(8) Logic Swing :-

→ difference of output voltage. $V_H - V_L$.

(9) Noise Immunity :-

→ The max. noise that can be present without causing any change in the output.

* Important points :-

→ The main problem of DCTL (Direct coupled transistor logic) is current hogging. To remove this problem, IIL (Integrated Injection logic) is used.

→ When among several parallel lines, one line draws more than the others; it is called current hogging.

→ CMOS has the best (highest) noise immunity

→ IIL (Integrated Injection logic) uses multiple-collector transistor, which gives a high packing density.

→ CMOS has the lowest power dissipation

→ RTL - Resistor transistor logic.

→ DTL - Diode " "

→ DCTL - Directed Couple " "

→ HTL - High threshold logic

→ TTL → transistor transistor logic
→ ECL - Emitted Coupled logic
→ PMOS - (P-channel metal oxide Semiconductor) (↑ voltage off)
→ NMOS - N-channel (↑ voltage, ON)
→ CMOS - Complementary. (P+N MOS)

* SOP and POS :-

SOP :-
→ Sum of product
→ (AB) + (BC) + (AC)
→ when fn, f = 1
→ A → 1
$\bar{A}$ → 0

→ Product of Sum.
→ (A+B) . (B+C) . (C+C)
→ when f = 0
A → 0
$\bar{A}$ → 1

	A	B	f
m_0	0	0	0
m_1	0	1	1
m_2	1	0	1
m_3	1	1	1

Express f in Sum of product form :-
When f = 1.
$0 = \bar{A}$
$1 = A$
⇒ $0\,1 + 1\,0 + 1\,1$
⇒ $\bar{A}B + A\bar{B} + AB$ → standard/ canonical form (Contain all val

SOP → 1 → min term

Σm = SOP
Πm = POS

$f(A,B) = m_1 + m_2 + m_3$
$f(A,B) = \Sigma m(1+2+3)$

Express f in product of sum.

	A	B	f
m_0	0	0	0
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

* When f = 0

0 = A

1 = $\bar{A}$

$f(A,B) = (0+0).(1+0)$

$= \underbrace{(A+B).(\bar{A}+B)}_{\text{maxterm.}}$

$f(A,B) = M_0 . M_2$

$f(A,B) = \pi_M(m_0, m_2) \Rightarrow f(A,B) = \pi_M(0,2)$ $(0,2)$

Ques:- Represent / Expand $f(A,B) = \bar{A} + \bar{B}$ in ~~SOP~~ SOP & POS. Also find minterm and maxterm.

Soln: $f(A,B) = \underbrace{\bar{A}}_{\text{B missing}} + \underbrace{\bar{B}}_{\text{A missing.}}$

$= \bar{A}(B+\bar{B}) + \bar{B}(A+\bar{A})$

$= \bar{A}B + \bar{A}\bar{B} + \bar{B}A + \bar{B}\bar{A}$

$= \bar{A}B + \bar{A}\bar{B} + \bar{B}A$

$= 01 + 00 + 10$

$= m_1 + m_0 + m_2$

$= \Sigma_m(0,1,2)$ minterm

* In minterm m_3 is missing, So M_3 will be maxterm.

Hence POS $= \pi_M(3)$

$= 11$

$= \bar{A} + \bar{B}$

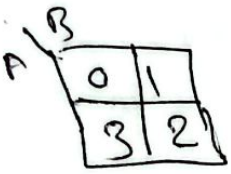
* K-map:

→ Karnaugh map.

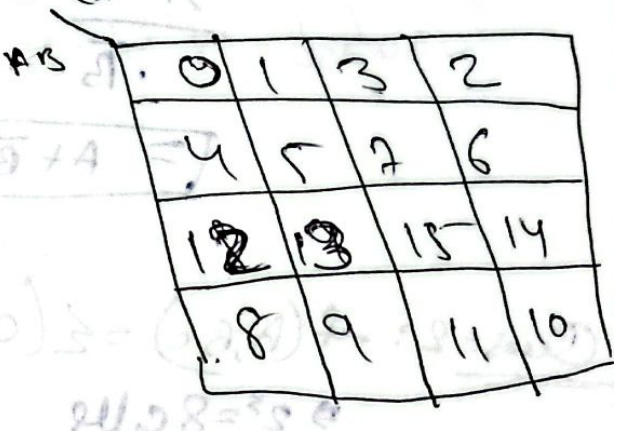
→ It is used to simply boolean expression in pictorial form without using law.

→ 2^n = , n is no. of variable.

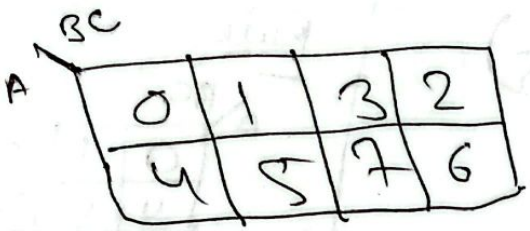
$2^2 = 4$:-



$2^4 = 16$:- CD



$2^3 = 8$.



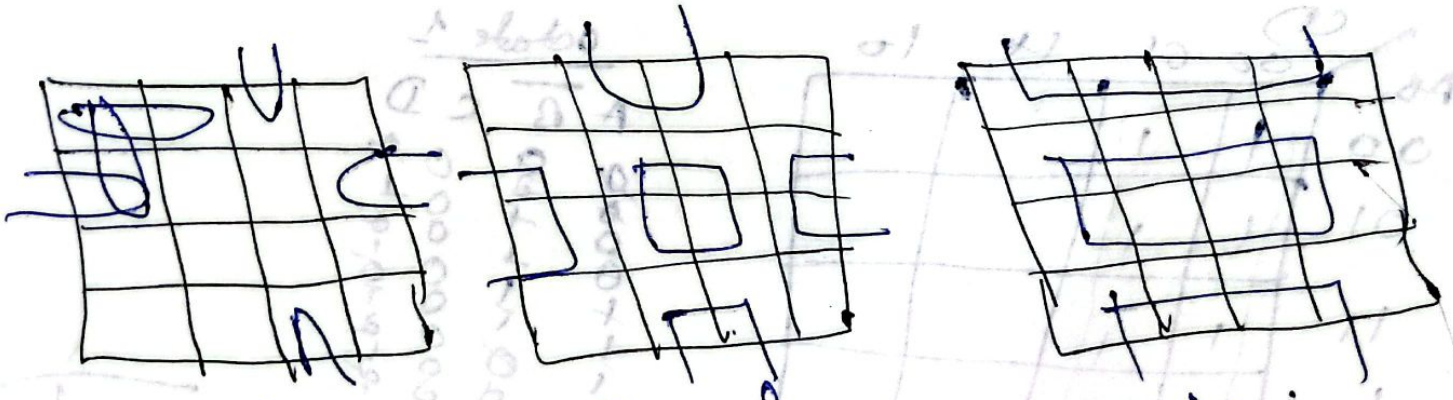
* Rules for creating group in K-map:-

→ No zero allowed.

→ Group can be Horizontal & vertical but not diagonal.

→ overlapping allowed

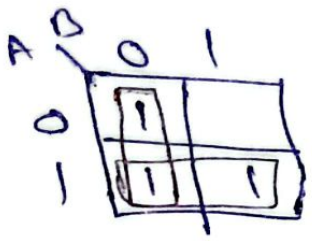
→ Group should be as large as possible.



Ques 1: $f(A,B) = \Sigma(0,2,3)$

⇒ 2 variable.

Cell $= 2^2 = 4$ cells.



Pair 1 | Pair 2

A B | A B

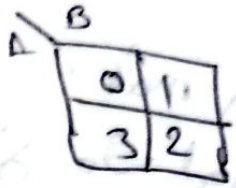
$\cancel{0}$ 0 | 1 $\cancel{0}$

$\cancel{1}$ 0 | 1 $\cancel{1}$

$\overline{B}$ | A

$= A + \overline{B}$.

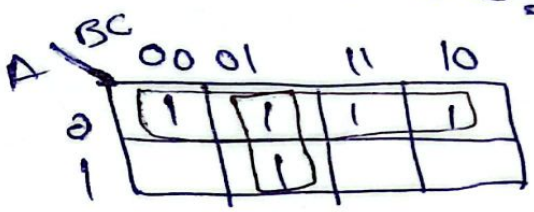
Rule ⇒



→ Same pair if 00 in Cell

Ques = 2: $f(A,B,C) = \Sigma(0,1,2,3,5)$

⇒ $2^3 = 8$ cells.



Quad 1 | Quad 2

A B C | A B C

0 $\cancel{0}$ $\cancel{0}$ | $\cancel{0}$ 0 1

0 $\cancel{1}$ 1 | 0 1

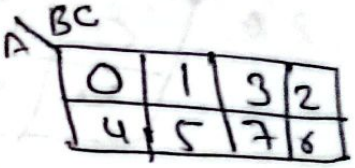
0 1 1

0 1 $\cancel{0}$

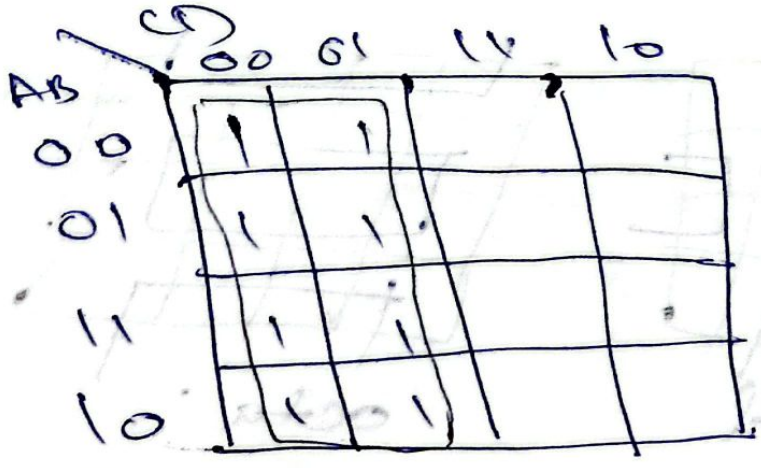
$\overline{A}$ | $\overline{B}$ C

$= \overline{A} + \overline{B}C$

Rule:



Ques = 3: $f(A,B,C,D) = \Sigma(0,1,4,5,8,9,12,13)$



Octade 1

A B C D

$\cancel{0}$ $\cancel{0}$ 0 $\cancel{0}$

$\cancel{0}$ $\cancel{1}$ 0 1

0 1 0 0

0 1 0 1

1 $\cancel{0}$ 0 0

1 $\cancel{0}$ 0 1

$\overline{C}$ ⇒ $\boxed{\overline{C}}$

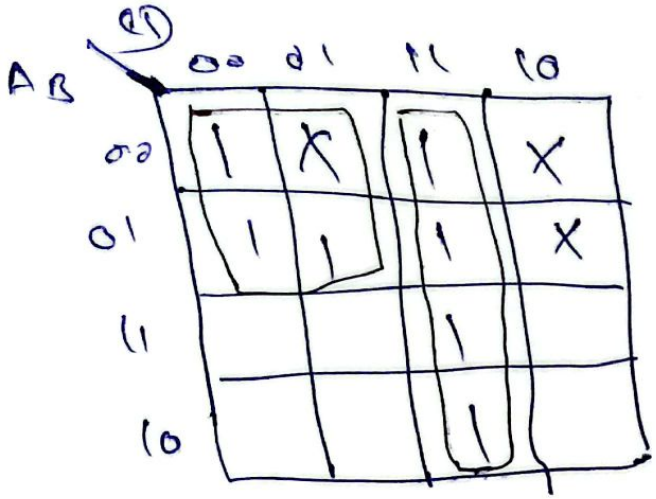
* Don't Care Situation:

→ When the output of a boolean expression is unspecified or the output is not needed, the condition is called don't care.

→ In k-maps, it denoted as X.

→ In such cases, while making groups, the largest possible groups are preferred and X's should not be included as much as possible.

Ex: $f(A,B,C,D) = (0,3,7,5,4,11,15)$
$d(A,B,C,D) = (1,2,6)$



Quad 1

A	B	C	D
0	0	0	0
0	0	0	1
0	1	0	0
0	1	0	1

$\bar{A}\,\bar{C}$

Quad 2

A	B	C	D
0	0	1	1
0	1	1	1
1	1	1	1
1	0	1	1

CD

$= \bar{A}\bar{C} + CD$